

【校招串讲】Kernel选择分发体系梳理与优化

目录

- 贾宏宇 2022.10.17
- 一、基本概念梳理
 - 1.1 以linear运算为例
 - 1.2 OP与kernel概念梳理
 - 1.2.1 静/老动态图的OP概念梳理（剪枝版）
 - 1.2.2 OP与kernel注册
 - 1.2.2.1 OP注册（无BLD信息）
 - 1.2.2.2 Kernel注册（有BLD信息）
- 二、老动态图调用逻辑梳理
 - 2.1 python端调用
 - 2.2 解析参数（代码生成）：解析input、解析 attr 键值对、占位 output；最终调用 TraceOp(...)
 - 2.3 构造OperatorBase（静态图）、attr属性检查、AMP类型转换；最终调用 OpBase::Run(...)
 - 2.4 设置outs数据类型、准备PreparedOp、准备Transform后数据；最终调用 PreparedOp::Run(...)
 - 2.4.1 Prepare函数（构造、与phi兼容）： GetExpectedKernelType、kernel选择、构造 PreparedOp对象；
 - 2.4.2 PrepareData函数（数据）： GetKernelTypeForVar准备输入数据（数据BLD转换）
 - 2.4.3 调用kernel（运算）： InferShape设置output，区分调用的是 fluid 还是 phi 下的 kernel
 - 2.5 总结
- 三、静态图调用逻辑梳理
- 四、新动态图调用逻辑梳理
 - 4.1 调用逻辑梳理
 - 4.1.1 python端调用
 - 4.1.2 解析参数（代码生成）：解析input、解析 attr、获取输出 output；数据类型Tensor
 - 4.1.3 dygraph层（代码生成）： AMP类型转换、attr属性检查（无）；数据类型Tensor

- 4.1.4 api层（代码生成）：Kernel选择（GetExpectedKernelType、GetKernelTypeForVar）、DataTransform、InferShape、Compute
- 4.2 对应关系
 - 4.2.1 新动态图中的对应关系
 - 4.2.2 GetExpectedKernelType的对应关系
 - 4.2.3 GetKernelTypeForVar的对应关系
- 4.3 总结
 - 4.3.1 优点（相比于静/老动态图）
 - 4.3.2 与静/老动态图的异同
 - 4.3.3 可能存在的问题
- 五、PyTorch dispatch调研
 - 5.1 PyTorch 中的 dispatch 机制
 - 5.1.1 初识 DispatchKey（优先级从低到高）
 - 5.1.2 DispatchKey 详谈
 - 5.2 dispatch 相关数据结构
 - 5.2.1 类关系描述
 - 5.2.2 OperatorEntry 类：记录算子对应的 dispatchTable_
 - 5.2.3 OperatorHandle/ TypedOperatorHandle 类：记录op信息，提供 schema 相关的查询，提供 kernel 的调用接口
 - 5.2.4 Dispatcher：组合上面的类功能，实现算子选择分发
 - 5.3 kernel 注册与调用方式
 - 5.3.1 dispatcher中的表结构
 - 5.3.2 注册kernel至dispatcher
 - 5.4 Paddle 与 PyTorch 的比较
- 六、Kernel 选择分发优化方案
 - 6.1 背景
 - 6.2 执行路径
 - 6.2.1 fluid 下的执行路径
 - 6.2.2 phi 下的执行路径
 - 6.3 硬编码优化方案
 - 6.3.1 代码梳理
 - 6.3.2 硬编码外移（已完成）
 - 6.3.3 硬编码下沉（进行中）
 - 6.4 新动态图支持 mkldnn kernel
 - 备选方案1
 - 其他方案（需持续调研）
- 七、个人工作和感悟
 - 7.1 个人工作

- 7.1.1 算子迁移与机制完善 (0718-0901)
- 7.1.2 kernel 选择分发相关工作 (0902 至今)
- 7.2 感悟
- 八、现场讨论问题整理与答复

贾宏宇 2022.10.17

一、基本概念梳理

1.1 以linear运算为例

从一个最为基础的全链接层开始介绍

</> paddle/nn/functional/common.py

Python | 收起 ^

```
1 # Out = XW + b
2 def linear(x, weight, bias=None, name=None):
3     if in_dygraph_mode(): # 新动态图
4         return _C_ops.linear(x, weight, bias)
5     else:
6         if _in_legacy_dygraph(): # 老动态图
7             pre_bias = _legacy_C_ops.matmul_v2(x, weight, 'trans_x', False,
8             'trans_y', False)
9             if bias is None:
10                 return pre_bias
11             return _legacy_C_ops.elementwise_add(pre_bias, bias)
12         else: # 静态图
13             .....
14             helper.append_op(type='matmul_v2',
15                               inputs=inputs,
16                               outputs={'Out': tmp},
17                               attrs=attrs)
18             .....
```

从外部用户的角度，调用最为常见的 `linear` 运算，可以观察到：

- 当前框架有新动态图、老动态图、静态图三种运算模式
- 老动态图中，`linear` 运算由 `matmul_v2` 和 `elementwise_add` 运算组合而成

从框架开发者的角度，思考如何实现 `matmul_v2` 运算：

- 数据（输入、输出）：数据存储在哪个硬件上？在内存中的组织形式如何？数据类型是什么？
 - Backend：CPU、ONEDNN、GPU、XPU、ALL_BACKEND.....

- **Layout**: NCHW、NDHWC、**ONEDNN**、SPARSE_COO、**ALL_LAYOUT**、KNCHW.....
- **Datatype**: BOOL、INT8、FLOAT32、COMPLEX64、**ALL_DTYPE**.....
- **计算（算子与kernel）**: 使用 `matmul_v2` 函数，如何处理不同类型的数据？
 - **算子**: 用户通用的计算接口，调用一个 `matmul_v2` 算子即可，BLD无关（BLD={backend, layout, datatype}
 - **kernel**: 内部特化的计算逻辑，适配多个 `matmul_v2` kernel，BLD相关
 - **kernel选择分发**: 连接两者之间的桥梁

1.2 OP与kernel概念梳理

如何定义一个OP? 运算的名称是什么、输入输出是什么

——运算名称: `matmul_v2`; 输入: `x` 和 `weight`; 输出: `output`

如何调用kernel? **kernel选择分发**, 贯穿了从OP注册到kernel调用的过程

1.2.1 静/老动态图的OP概念梳理（剪枝版）

前向op

`OpMaker` # 构造算子，描述算子的输入、输出和属性

`OperatorWithKernel` # 算子类，包括从OP如何运行到kernel（几个重点概念）

- `InferShape` # 算子输出形状的推导函数

- `GetExpectedKernelType` # 算子需要的Kernel类型（kernel选择分发——通用选择）

- `GetKernelTypeForVar` # 某个Var特定的Kernel类型（kernel选择分发——特化选择）

Op注册

`REGISTER_OPERATOR` # 将算子注册到全局map中（模版元编程）

</> OP——paddle/fluid/operators/matmul_v2_op.cc

C++ | 收起 ^

```
1 class MatMulV2OpMaker : public framework::OpProtoAndCheckerMaker {
2 public:
3     void Make() override {
4         AddInput("X", "tensor of shape (d0, d1 ... M, K)");
5         AddInput("Y", "tensor of shape (d0, d1 ... K, N)");
6         AddOutput("Out", "tensor of shape (d0, d1 ... M, N)");
7         // 添加属性的同时，也会添加 AddAttrChecker，进行属性检查
8         AddAttr<bool>("trans_x").SetDefault(false);
9         AddAttr<bool>("trans_y").SetDefault(false);
10    }
11 };
12
```

```
13 class MatMulV2Op : public framework::OperatorWithKernel {
14 public:
15     void InferShape(framework::InferShapeContext* ctx) const override {
16         ... }
17     protected:
18         framework::OpKernelType GetExpectedKernelType(const
19             framework::ExecutionContext& ctx) const override { ... }
20         framework::OpKernelType GetKernelTypeForVar(const std::string&
21             var_name, const phi::DenseTensor& tensor, const framework::OpKernelType&
22             expected_kernel_type) const override { ... }
23 };
24
25 REGISTER_OPERATOR(matmul_v2,
26     // 算子名称
27     ops::MatMulV2Op,
28     // 算子类, 分发信息
29     ops::MatMulV2OpMaker,
30     // 算子定义信息
31     ops::MatMulV2GradOpMaker<paddle::framework::OpDesc>,
32     // 反向算子定义信息, 静态图
33     ops::MatMulV2GradOpMaker<paddle::imperative::OpBase>);
34 // 反向算子定义信息, 动态图
```

1.2.2 OP与kernel注册

1.2.2.1 OP注册（无BLD信息）

全部在类的构造函数完成：

- 模版元编程 --> 模版结构体（登记员） --> 递归注册op和信息（OpInfo 保存了 op 和反向的信息）

</> 注册代码

C++ | 收起 ^

```
1 REGISTER_OPERATOR(matmul_v2,
2     ops::MatMulV2Op,
3     ops::MatMulV2OpMaker,
4     ops::MatMulV2GradOpMaker<paddle::framework::OpDesc>,
5     ops::MatMulV2GradOpMaker<paddle::imperative::OpBase>);
```

</> 第一次展开

C++ | 收起 ^

```
1 // 检测当前处于全局命名空间
2 STATIC_ASSERT_GLOBAL_NAMESPACE( __reg_op__matmul_v2, "REGISTER_OPERATOR
3     must be called in global namespace");
4 // 登记员模版结构体, 注册op至全局单例
```

```

4 static ::paddle::framework::OperatorRegistrar<
5     ops::MatMulV2Op,
6     ops::MatMulV2OpMaker,
7     ops::MatMulV2GradOpMaker<paddle::framework::OpDesc>,
8     ops::MatMulV2GradOpMaker<paddle::imperative::OpBase>
9     > __op_registrar_matmul_v2__("matmul_v2");
10 // 构造函数中注册op，需要有函数调用，否则链接器生成的二进制文件会删除这部分动作
11 int TouchOpRegistrar_matmul_v2() { __op_registrar_matmul_v2__.Touch();
    return 0; }

```

</> paddle/fluid/framework/op_registry.h

C++ | 收起 ^

```

1 template <typename... ARGS>
2 struct OperatorRegistrar : public Registrar {
3     explicit OperatorRegistrar(const char* op_type) {
4         OpInfo info;
5         details::OperatorRegistrarRecursive<0, false, ARGS...>(op_type,
6             &info);
7         OpInfoMap::Instance().Insert(op_type, info);    // 全局单例
8     }
9 };
10 // 递归注册 OP 信息
11 template <size_t I, typename... ARGS>
12 class OperatorRegistrarRecursive<I, false, ARGS...> {
13 public:
14     using T = typename std::tuple_element<I, std::tuple<ARGS...>>::type;
15     OperatorRegistrarRecursive(const char* op_type, OpInfo* info) {
16         OpInfoFiller<T> fill;    // 匹配到对应的注册类型
17         fill(op_type, info);
18         constexpr auto size = sizeof...(ARGS);    // 【优化】不需要根据 size 做
19         // 判断，ARGS 可以自动展开
20         OperatorRegistrarRecursive<I + 1, I + 1 == size, ARGS...>
21         reg(op_type, info);
22         (void)(reg);
23     }
24 };
25 // fill 以函数指针的形式，注册下列信息（info 与 op 通过 op 名来连接）
26 // info中没有注册 GetExpectedKernelType 和 GetKernelTypeForVar
27 struct OpInfoFiller<T, kOperator>    // 注册 info->creator_
    和 info->infer_shape_
28 struct OpInfoFiller<T, kShapeInference>    // 注册 info-
    >infer_shape_（新动态图覆盖老动态图）
29

```

```
28 DECLARE_INFER_SHAPE_FUNCTOR(bilinear_interp_v2,  
29                               BilinearInterpInferShapeFunctor,  
30                               PD_INFER_META(phi::InterpolateInferMeta))  
31  
32 // 【省略】其他注册类型，时间原因不再展开  
33 struct OpInfoFiller<T, kOpProtoAndCheckerMaker>    // 注册 info->proto_  
    和 info->checker_  
34 struct OpInfoFiller<T, kGradOpDescMaker>          // 注册 info-  
    >grad_op_maker_和使用default、empty grad的标志  
35 struct OpInfoFiller<T, kGradOpBaseMaker>          // 注册 info-  
    >dygraph_grad_op_maker_  
36 struct OpInfoFiller<T, kVarTypeInference>         // 注册 info-  
    >infer_var_type_  
37 struct OpInfoFiller<T, kInplaceOpInference>       // 注册 info-  
    >infer_inplace_  
38 struct OpInfoFiller<T, kNoNeedBufferVarsInference> // 注册 info-  
    >infer_no_need_buffer_vars_  
39 struct OpInfoFiller<void, kUnknown>               // 无注册逻辑
```

1.2.2.2 Kernel注册（有BLD信息）

</> Kernel——paddle/fluid/operators/matmul_op.cc

C++ | 收起 ^

```
1 template <typename DeviceContext, typename T>  
2 class MatMulKernel : public framework::OpKernel<T> {  
3     public:  
4         void Compute(const framework::ExecutionContext &context) const  
            override { ... }  
5 };  
6  
7 REGISTER_OP_KERNEL(matmul,  
8                     CPU,  
9                     ::paddle::platform::CPUTPlace,  
10                    ops::MatMulKernel<phi::CPUContext, float>,  
11                    ops::MatMulKernel<phi::CPUContext, double>);
```

最终通过函数注册kernel:

- 登记员仿函数递归解析 --> RegisterKernelClass 函数注册 kernel

</> paddle/fluid/framework/op_registry.h

C++ | 收起 ^

```
1 static ::paddle::framework::OpKernelRegistrar<          // 通过模版参数传递的  
    信息  
2     ::paddle::platform::CPUTPlace,                      // place  
3     ops::MatMulKernel<phi::CPUContext, float>,          // datatype
```

```
4 ops::MatMulKernel<phi::CPUContext, double> // datatype
5 > __op_kernel_registrar_matmul_CPU_DEFAULT_TYPE__(// 通过构造函数参数传
    递的信息
6 "matmul", // op 名称
7 "CPU", // library_type
8 ::paddle::framework::OpKernelType::kDefaultCustomizedTypeValue);
// custom_value
9
10
11 template <typename PlaceType, typename... KernelType>
12 class OpKernelRegistrar : public Registrar {
13 public:
14     explicit OpKernelRegistrar(const char* op_type, const char*
        library_type, int customized_type_value) {
15         // 仿函数构造 kernel
16         OpKernelRegistrarFunctor<PlaceType, false, 0, KernelType...> func;
17         func(op_type, library_type, customized_type_value);
18     }
19 };
20
21
22 template <typename PlaceType, size_t I, typename... KernelTypes>
23 struct OpKernelRegistrarFunctor<PlaceType, false, I, KernelTypes...> {
24     void operator()(const char* op_type, const char* library_type, int
        customized_type_value) const {
25         // 通过 RegisterKernelClass 函数将 kernel_type 和 func 注入全局单例中
26         RegisterKernelClass<PlaceType, T>(op_type, library_type,
            customized_type_value,
27         // 将类中的 Compute 函数, 封装为函数指针
28         [op_type](const framework::ExecutionContext& ctx) {
29             KERNEL_TYPE().Compute(ctx);
30             CheckKernelLaunch<PlaceType>(op_type);
31         });
32         constexpr auto size =
            std::tuple_size<std::tuple<KernelTypes...>>::value;
33         OpKernelRegistrarFunctor<PlaceType, I + 1 == size, I + 1,
            KernelTypes...> func;
34         func(op_type, library_type, customized_type_value);
35     }
36 };
```

</> RegisterKernelClass函数

C++ | 收起 ^

```
1 template <typename PlaceType, typename T, typename Func>
```

```
2 inline void RegisterKernelClass(const char* op_type,
3                               const char* library_type,
4                               int customized_type_value,
5                               Func func) {
6     std::string library(library_type);
7     std::string data_layout = "ANYLAYOUT";           // 老动态图默认注册所有的
    layout! 实际上不根据 layout 做选择
8     if (library == "MKLDNN") {                     // 此处的data_layout实际上是
        是和mkldnn绑定的
9         data_layout = "MKLDNNLAYOUT";
10    }
11    OpKernelType key(ToDataType(std::type_index(typeid(T))),
12                    PlaceType(),
13                    StringToDataLayout(data_layout),
14                    StringToLibraryType(library_type),
15                    customized_type_value);
16    OperatorWithKernel::AllOpKernels()[op_type][key] = func;
17 }
```

- ✓ 新动态图的OP信息如何组织？Kernel如何选择？
——后文对比新动态图和静态图中概念的异同

二、老动态图调用逻辑梳理

📘 本次串讲详细梳理新/老动态图的调用逻辑（老动态图即将被废弃，为什么还需要详细介绍？）：

- 老动态图的调用过程，揉合了静态图、phi的逻辑
 - 老动态图**复用静态图**的部分实现
 - 老动态图**优先调用phi**下的kernel
- Kernel选择分发体系的目标
 - 去除静态图 GetExpectedKernelType 中的硬编码时，仍需支持老动态图的调用逻辑
 - 新动态图选择分发 mkldnn kernel 时，希望统一新老动态图的分发逻辑

注：为了介绍的简洁，部分示例代码有删减，仅展示代码中的重要逻辑

构建一个静态图的OperatorBase --> 使用动态图的OpBase调用静态图op --> PreparedOp执行真实的计算逻辑

2.1 python端调用

</> python端调用API——matmul_v2

Python | 收起 ^

```
1 # python/paddle/nn/functional/common.py
```

```
2 if _in_legacy_dygraph():
3     pre_bias = _legacy_C_ops.matmul_v2(x, weight, 'trans_x', False,
        'trans_y', False)
```

2.2 解析参数（代码生成）：解析input、解析 attr 键值对、占位 output；最终调用 TraceOp(...)

</> C++函数的开始——imperative_matmul_v2

C++ | 收起 ^

```
1 // paddle/fluid/pybind/op_function1.cc
2 static PyObject * imperative_matmul_v2(PyObject *self, PyObject *args,
    PyObject *kwargs)
3 {
4     std::string op_type = "matmul_v2";
5     // 返回类型: std::shared_ptr<imperative::VarBase>
6     auto X = GetVarBaseFromArgs(op_type, "X", args, 0, false);
7     auto Y = GetVarBaseFromArgs(op_type, "Y", args, 1, false);
8     // std::unordered_map<std::string, Attribute>
9     framework::AttributeMap attrs;
10    // 属性按照 {attr_name: data_type} 组织，解析出一个参数需要遍历两次
11    ConstructAttrMapFromPyArgs(op_type, args, 2, PyTuple_GET_SIZE(args)
        , attrs);
12    // 保存输入和输出参数，Out参数的VarBase名称使用atomic，不会重复
13    // 此处只是声明了一个out变量，规定类型和分配空间在 OpBase::Run 函数中
14    // std::map<std::string, std::vector<std::shared_ptr<VarBase>>>
15    imperative::NameVarBaseMap outs = {"Out",
        {std::shared_ptr<imperative::VarBase>(new
            imperative::VarBase("auto_"+std::to_string(VarBaseUniqueNameID++)+"_"
        ))}}};
16    imperative::NameVarBaseMap ins = {"X", {X}}, {"Y", {Y}};
17    // 关键调用函数
18    imperative::GetCurrentTracer()->TraceOp(op_type, ins, outs, attrs,
        {});
19    // 返回结果
20    return MakeReturnPyObject(outs["Out"][0]);
21 }
```

2.3 构造OperatorBase（静态图）、attr属性检查、AMP类型转换；最终调用 OpBase::Run(...)

</> Tracer::TraceOpImpl——运算逻辑

C++ | 收起 ^

```
1 // paddle/fluid/imperative/tracer.cc & tracer.h
2 template <typename VarType>
```

```

3 void Tracer::TraceOpImpl( ... ) {
4     .....
5     if (FLAGS_use_mkldnn) { // 全局bool标志
6         // 1. 根据 FLAGS_tracer_mkldnn_ops_on/off, 修改attrs["use_mkldnn"]属性
7     }
8     // 2. 复用静态图逻辑, 构造OP, CreateOp 的形参为 VariableNameMap, 无法传入
9     OperatorBase op = framework::OpRegistry::CreateOp(type, {}, {}, {},
10    false);
11    // 3. 【省略】attr 属性检查
12    auto* attr_checker = op->Info().Checker();
13    // 4. 【省略】获取默认参数
14    const paddle::framework::AttributeMap& default_attrs = /*...*/
15    attr_checker->GetDefaultAttrMap()
16    // 5. 【省略大段代码】AMP 模式下对 inputs 做转换
17    // 6. 【省略】根据 GPU、XPU、NPU、MLU 设置设备ID
18    platform::SetDeviceId(place.device);
19    // 7. 旧动态图兼容静态图, OperatorBase是静态图的实现
20    OpBase::Run(*op, new_ins, outs, attrs, default_attrs, place);
21    // 8. 【省略】旧动态图相关逻辑
22    if (ComputeRequiredGrad(new_ins, outs, trace_backward)) {
23        CreateGradOpNode(
24            *op, new_ins, outs, attrs, default_attrs, place, inplace_map);
25    }
26 }

```

2.4 设置outs数据类型、准备PreparedOp、准备Transform后数据；最终调用 PreparedOp::Run(...)

class PreparedOp的成员函数极其简单，只有**Prepare**函数（构造PreparedOp）、**Run**函数

</> OpBase::Run & OpBaseRunImpl

C++ | 收起 ^

```

1 // paddle/fluid/imperative/layer.cc
2 template <typename VarType>
3 static void OpBaseRunImpl( ... ) {
4     // 1. 基类转派生类, 象征着 kernel 选择分发的开始
5     auto* op_kernel = static_cast<const
6     framework::OperatorWithKernel*>(&op);
7     if (info.infer_var_type_) {
8         // NOTE: 此处是 infer_var_type_ 推理 var 的shape, 推导 outs 的数据类型, 最
9         终会调用 SetVarDataType
10        RuntimeInferVarTypeContext<VarType> infer_var_type_ctx(ins, outs,
11        attrs, default_attrs);
12        info.infer_var_type_(&infer_var_type_ctx);
13    }
14 }

```

```

10  }
11  // 2. 【省略】设置 outs 的数据类型
12
13  // 3. 准备 op ，此处进行 kernel 选择，输入的 kernel 类型为
    OperatorWithKernel
14  auto prepared_op = PreparedOp::Prepare(ins, outs, *op_kernel, place,
    attrs, default_attrs);
15  // 4. 准备输入参数，将 ins 参数转换为 prepared_op.kernel_type()
16  auto tmp_ins_ptr = PrepareData<VarType>(*op_kernel, ins,
    prepared_op.kernel_type());
17  // 5. 准备好 op 和 数据之后，终于可以执行了!!!
18  prepared_op.Run(*tmp_ins_ptr, outs, attrs, default_attrs);
19
20  // 将 var->grad 的 type 设置为 output var 的 type
21  SetForwardDataTypeOfGradVars<VarType>(outs);
22 }

```

2.4.1 Prepare函数（构造、与phi兼容）：GetExpectedKernelType、kernel选择、构造PreparedOp对象；

kernel 选择的优先级：phi Xpu kernel > fluid Xpu kernel > phi cpu kernel > fluid cpu kernel

</> PreparedOp::Prepare & PrepareImpl

C++ | 收起 ^

```

1  // paddle/fluid/imperative/prepared_operator.cc
2
3  // kernels_: 二层map: {string kernel_name: {KernelKey key: Kernel}};
    // kernel 的全局单例
4  const phi::KernelFactory& PreparedOp::phi_kernel_factory
5  // base_kernel_name_map_: {string old_name, string new_name}
    // 函数名称的映射
6  // arg_mapping_fn_map_: {string name, ArgumentMappingFn func}
    // 旧动态图的参数
7  const phi::OpUtilsMap& PreparedOp::phi_op_utils_map
8  // map_: {string, KernelSignature sig}
    // 函数签名
9  const phi::DefaultKernelSignatureMap&
    PreparedOp::default_phi_kernel_sig_map
10
11  // 【优化】一个函数300多行代码，亟需优化
12  template <typename VarType>
13  PreparedOp PrepareImpl( ... ) {
14  // 1. GetExpectedKernelType: 获得期望运行的 Kernel 类型
15  auto dygraph_exe_ctx = DygraphExecutionContext<VarType>(op,
    empty_scope, *dev_ctx, empty_ctx, ins, outs, attrs, default_attrs);

```

```

16 auto expected_kernel_key = op.GetExpectedKernelType(dygraph_exe_ctx);
17
18 // 2. 根据 sig 的有无判断是否有 phi 的kernel
19 bool has_phi_kernel = false;
20 const auto* arg_map_fn =
    phi_op_utils_map.GetArgumentMappingFn(op.Type());
21 if (arg_map_fn) {
22     has_phi_kernel = true;
23     kernel_signature = (*arg_map_fn)
        (framework::ExecutionArgumentMappingContext(dygraph_exe_ctx));
24 }
25 // 3. 【省略大段代码】kernel选择, 从全局的 phi_kernel_factory 进行两次查找
26 // kernel 选择的优先级: phi Xpu kernel > fluid Xpu kernel > phi cpu
    kernel > fluid cpu kernel
27 // 4. 根据 has_phi_kernel 判断调用 phi 还是 fluid 的构造函数
28 // has_phi_kernel 的构造函数, 内部设置了run_phi_kernel_
29 return PreparedOp(op,                                // OperatorBase
30                  empty_ctx,                            // RuntimeContext
31                  expected_kernel_key,                  // OpKernelType
32                  arg_map_fn,                          //
    ArgumentMappingFn
33                  default_kernel_signature,            //
    KernelSignature
34                  std::move(kernel_signature),          //
    KernelSignature
35                  phi_kernel,                          // Kernel
36                  dev_ctx);                            // DeviceContext
37 // fluid 的构造函数
38 return PreparedOp(op,
39                  empty_ctx,
40                  expected_kernel_key,
41                  kernel_iter->second, //
    OperatorWithKernel::OpKernelFunc, void(const ExecutionContext&)
42                  arg_map_fn,
43                  default_kernel_signature,
44                  dev_ctx);
45 }

```

2.4.2 PrepareData函数 (数据): GetKernelTypeForVar准备输入数据 (数据BLD转换)

</> PrepareData

C++ | 收起 ^

```

1 // paddle/fluid/imperative/prepared_operator.h
2 template <typename VarType>

```

```

3  std::shared_ptr<NameVarMap<VarType>> PrepareData(
4      const framework::OperatorWithKernel& op,
5      const NameVarMap<VarType>& ins,
6      const framework::OpKernelType& expected_kernel_key) {
7      std::shared_ptr<NameVarMap<VarType>> tmp_ins_ptr = nullptr;
8      for (const auto& name_pair : ins) {
9          for (size_t i = 0; i < name_pair.second.size(); ++i) { // 处理每个
Tensor
10             if (tensor && tensor->IsInitialized() && (tensor->memory_size() !=
0)) {
11                 // 1. 调用 GetKernelTypeForVar 函数, 对每个 Var 特化进行 kernel 选择
12                 // 默认按照 {expected_kernel_type.data_type_, tensor.place(),
tensor.layout()} 组织
13                 auto kernel_type_for_var =
op.GetKernelTypeForVar(name_pair.first, *tensor, expected_kernel_key);
14                 // 2. BLD 均相同, 不需要转换
15                 if (!NeedTransform(kernel_type_for_var, expected_kernel_key)) {
16                     continue;
17                 } else { // 3. 如果var的kernel_type和期待的不一致,
kernel_type_for_var -> expected_kernel_key
18                     phi::DenseTensor out;
19                     // 转换 layout、datatype、device
20                     TransformData(expected_kernel_key, kernel_type_for_var,
*tensor, &out);
21 }}}}
```

2.4.3 调用kernel (运算): InferShape设置output, 区分调用的是 fluid 还是 phi 下的 kernel

</> PreparedOp::Run

C++ | 收起 ^

```

1  void PreparedOp::Run(const NameVarMap<VarBase>& ins,
2                      const NameVarMap<VarBase>& outs,
3                      const framework::AttributeMap& attrs,
4                      const framework::AttributeMap& default_attrs) {
5      // 根据 run_phi_kernel_ 标志判断用新/老动态图
6      if (run_phi_kernel_) {
7          PreparedOpRunPtImpl<VarBase>(op_, kernel_type_, arg_map_fn_,
default_kernel_signature_,
8                                         kernel_signature_, phi_kernel_,
9                                         dev_ctx_, ins, outs, attrs,
default_attrs);
10         // op.Info().infer_shape(&infer_shape_ctx);
11         // PreparePhiData<VarType>(phi_kernel, kernel_signature, ins);
```

```

12    // phi_kernel(&phi_kernel_context);
kernel 计算
13    // 【省略】其他逻辑
14 } else {
15     PreparedOpRunImpl<VarBase>(op_, ctx_, kernel_type_, func_,
arg_map_fn_, default_kernel_signature_,
16                               dev_ctx_, ins, outs, attrs,
default_attrs);
17    // op.Info().infer_shape(&infer_shape_ctx);
18    // func(DygraphExecutionContext<VarType>(op, empty_scope, *dev_ctx,
ctx, ins, outs, attrs, default_attrs));
19    // 【省略】其他逻辑
20 }
21 }

```

2.5 总结

- 老动态图的前后向兼容
 - 前向：复用静态图构造OP的代码，节省了op迁移的成本
 - 后向：优先选择phi下的kernel
- 多个OP概念与数据处理逻辑相互穿插
 - 对于数据的处理，遍布整个调用流程：解析参数、参数检查、设置输出类型、DataTransform、设置输出形状。
 - 【优化】部分实现能否整合在一个函数中？
 - OperatorBase（静态图）、OpBase（老动态图）、PreparedOp（兼容新老动态图）
- GetKernelTypeForVar相比于GetExpectedKernelType，是更加特化的一种kernel 选择方式
- 历史背景：需要承前启后，代码为了兼容性牺牲了部分可读性，理解起来最为复杂

三、静态图调用逻辑梳理

- ❗ kernel选择分发子项，不依赖于对静态图组网等过程的调研，因此本章仅概述op的调用流程

此处假设已经调用完**CreateOp**，创建完成OperatorWithKernel类型的op，即将调用kernel进行运算（相当于老动态图2.4的调用逻辑）

注：为了介绍的简洁，部分示例代码有删减，仅展示代码中的重要逻辑

kernel 选择的优先级： phi Xpu kernel > fluid Xpu kernel > phi cpu kernel > fluid cpu kernel

```
</> OperatorWithKernel::RunImpl
```

C++ | 收起 ^

```

1 // paddle/fluid/framework/operator.cc
2 void OperatorWithKernel::RunImpl(const Scope& scope,
3                                  const platform::Place& place,

```

```
4         RuntimeContext* runtime_ctx) const {
5     // 相当于老动态图 2.4.1 章节, kernel 选择
6     // 静态图根据 op 名称判断是否有 phi kernel, 更新 phi 所需的一系列信息
7     if (phi::KernelFactory::Instance().HasCompatiblePhiKernel(type_)) {
8         kernel_signature_.reset(new
9 phi::KernelSignature(std::move(GetExpectedPhiKernelArgs(exe_ctx)))); //
函数签名
10         phi_kernel_name = kernel_signature_>name;
// kernel 名称
11         // 额外解析 Attr<std::string>("op_device"), 返回 OpKernelType
kernel_type_.reset(new
OpKernelType(std::move(InnerGetExpectedKernelType(exe_ctx)))); //
kernel 类型
12         phi_kernel_key =
TransOpKernelTypeToPhiKernelKey(*kernel_type_.get());
// kernel 类型
13         phi_kernel_.reset(new
phi::Kernel(phi::KernelFactory::Instance().SelectKernel(phi_kernel_name,
phi_kernel_key)));
14         run_phi_kernel_ = true;
15     }
16     // 选择 fluid 下的 kernel
17     if (!run_phi_kernel_) {
18         // 更新 kernel_type_ 和 kernel_func_, 如果没有找到 Xpu 下的 fluid
kernel, 则会 fallback 到 cpu
19         ChooseKernel(exe_ctx);
20     }
21
22     // 相当于老动态图 2.4.2 章节
23     // 【注】PrepareData 与老动态图的函数同名, 但是函数签名和逻辑不同, 需要用到静态图
的 scope 特性, 插入 Transform scope
24     std::vector<std::string> transfered_inplace_vars;
25     Scope* transfer_scope = nullptr;
26     transfer_scope = PrepareData(scope, *kernel_type_,
&transfered_inplace_vars, runtime_ctx);
27
28     // 相当于老动态图 2.4.3 章节
29     // InferShape设置output
30     this->Info().infer_shape(&infer_shape_ctx);
31     // kernel 调用逻辑
32     if (run_phi_kernel_) {
33         phi::KernelContext phi_kernel_context;
34         // 【此处省略 cache 优化】
35         phi::KernelContext phi_kernel_context;
```

```
36     BuildPhiKernelContext(*runtime_ctx, dev_ctx, &phi_kernel_context);
37     (*phi_kernel_)(&phi_kernel_context);
38 } else {
39     (*kernel_func_)(ExecutionContext(*this, exec_scope, *dev_ctx,
40     *runtime_ctx));
41 }
42 // 【省略】后续还有一些其他逻辑
43 }
```

四、新动态图调用逻辑梳理

- ✓ 新动态图中的op为函数式op，相比于老动态图，新动态图的调用路径更为简洁清晰、调度过程更为高效

4.1 调用逻辑梳理

4.1.1 python端调用

</> python端调用API——matmul

Python | 收起 ^

```
1 # python/paddle/tensor/math.py
2 if in_dygraph_mode():
3     return _C_ops.matmul(input, mat2, False, False)
4 elif paddle.in_dynamic_mode():
5     return _legacy_C_ops.matmul_v2(input, mat2)
```

4.1.2 解析参数（代码生成）：解析input、解析 attr、获取输出 output；数据类型 Tensor

</> C++函数的开始——eager_api_matmul

C++ | 收起 ^

```
1 // paddle/fluid/pybind/eager_op_function.cc
2 static PyObject * eager_api_matmul(PyObject *self, PyObject *args,
3 PyObject *kwargs) {
4     // 1. 解析输出，返回值为 Tensor
5     auto x = GetTensorFromArgs("matmul", "x", args, 0, false);
6     auto y = GetTensorFromArgs("matmul", "y", args, 1, false);
7     // 2. 【性能优化】直接根据 kernel 签名解析 attribute，而非解析键值对
8     PyObject* transpose_x_obj = PyTuple_GET_ITEM(args, 2);
9     bool transpose_x = CastPyArg2Boolean(transpose_x_obj, "matmul", 2);
10    PyObject* transpose_y_obj = PyTuple_GET_ITEM(args, 3);
11    bool transpose_y = CastPyArg2Boolean(transpose_y_obj, "matmul", 3);
12 }
```

```

12 // 设置设备ID, 仅支持GPU
13 auto place = egr::Controller::Instance().GetExpectedPlace();
14 phi::backends::gpu::SetDeviceId(place.device);
15
16 // 3. 调用 dygraph 层获得输出
17 decltype(::matmul_ad_func(x,y,transpose_x,transpose_y)) out =
  ::matmul_ad_func(x,y,transpose_x,transpose_y);
18 }

```

4.1.3 dygraph层（代码生成）：AMP类型转换、attr属性检查（无）；数据类型 Tensor

</> matmul_ad_func

C++ | 收起 ^

```

1 //
  paddle/fluid/eager/api/generated/eager_generated/forwards/dygraph_functions.cc
2 paddle::experimental::Tensor matmul_ad_func(const
  paddle::experimental::Tensor& x, const paddle::experimental::Tensor& y,
  bool transpose_x, bool transpose_y) {
3   // 【省略】ins 参数 AMP 类型转换
4   // 【省略】Layout autotune, 特定 op 专用逻辑
5
6   // Forward API Call, 调用的核心逻辑
7   auto api_result = paddle::experimental::matmul(x, y, transpose_x,
  transpose_y);
8
9   // 【省略大段代码】动态图构建和反向传播
10 }

```

4.1.4 api层（代码生成）：Kernel选择（GetExpectedKernelType、GetKernelTypeForVar）、DataTransform、InferShape、Compute

六次kernel选择逻辑：GPUDNN > GPUDNN_all_layout > kernel_backend > kernel_backend_all_layout > CPU > CPU_all_layout

</> matmul

C++ | 收起 ^

```

1 // paddle/phi/api/lib/api.cc
2 PADDLE_API Tensor matmul(const Tensor& x, const Tensor& y, bool
  transpose_x, bool transpose_y) {
3   // 1. 相当于老动态图下的 GetExpectedKernelType
4   // 首先将所有输入的信息放到一个 KernelKeySet 中
5   auto kernel_key_set = ParseKernelKeyByInputArgs(x, y);

```

```

6 // 然后, 根据 KernelKeySet 获取优先级最高的 KernelKey = {Backend,
  DataLayout, DataType}
7 auto kernel_key = kernel_key_set.GetHighestPriorityKernelKey();
8
9 // 2. kernel 选择【此处可优化, 依赖于准确指定layout】【与静/老动态图选择逻辑有差异】
10 auto kernel_result =
  phi::KernelFactory::Instance().SelectKernelOrThrowError(
11     "matmul", {kernel_backend, kernel_layout, kernel_data_type});
12 const auto& kernel = kernel_result.kernel;
13 auto* dev_ctx =
  GetDeviceContextByBackend(kernel_result.has_fallback_cpu ? Backend::CPU
    : kernel_backend);
14
15 // 3. 与静/老动态图的功能类似 PrepareData , 具有 GetKernelTypeForVar 和
  TransformData 的逻辑
16 auto input_x = PrepareData(x, kernel.InputAt(0), {});
17 auto input_y = PrepareData(y, kernel.InputAt(1), {});
18
19 // 4. InferShape 逻辑, 设置 output 形状
20 Tensor api_output;
21 auto kernel_out = SetKernelOutput(&api_output);
22 phi::MetaTensor meta_out(kernel_out);
23 phi::MatmulInferMeta(MakeMetaTensor(*input_x),
  MakeMetaTensor(*input_y), transpose_x, transpose_y, &meta_out);
24
25 // 5. 调用 kernel
26 using kernel_signature = void (*)(const platform::DeviceContext&,
27     const phi::DenseTensor&,
28     const phi::DenseTensor&,
29     bool,
30     bool,
31     phi::DenseTensor*);
32 auto* kernel_fn = kernel.GetVariadicKernelFn<kernel_signature>();
  // 找到 kernel function
33 (*kernel_fn)(*dev_ctx, *input_x, *input_y, transpose_x, transpose_y,
  kernel_out);
34 return api_output;
35 }

```

4.2 对应关系

✅ 在前向兼容的过程中, 原有逻辑不会消失, 无非是逻辑迁移或者逻辑覆盖

算子迁移时的每个步骤，究竟是在干什么？
phi是函数式kernel，实际上只有kernel，没有算子的概念。或者说yaml文件中的描述对应着 fluid 中的算子

4.2.1 新动态图中的对应关系

- # 前向op
- OpMaker--yaml函数签名 # 构造算子，描述算子的输入、输出和属性
- OperatorWithKernel--phi中为函数式概念 # 算子类，包括从OP如何运行到kernel
- InferShape--InferMeta # 算子输出形状的推导函数
- GetExpectedKernelType--yaml中的data_type # 算子需要的Kernel类型（kernel选择分发）
- GetKernelTypeForVar--phi中op注册时添加 #某个Var特定的Kernel类型（kernel选择分发）
- # Op注册
- REGISTER_OPERATOR # 将算子注册到全局map中（模版元编程）

</> paddle/phi/api/yaml/legacy_ops.yaml YAML | 收起 ^

```
1 - op : matmul # 与 fluid 中的 op 对应
2   args : (Tensor x, Tensor y, bool transpose_x = false, bool transpose_y = false) # 与 OpMaker 对应
3   output : Tensor
4   infer_meta : # 与 InferShape 对应
5     func : MatmulInferMeta
6     data_type : ..... # 指定函数的输入
7   kernel :
8     func : matmul # 调用的 kernel 函数
9     data_type : ..... # GetExpectedKernelType 根据某个输入返回结果
10  backward : matmul_grad
```

4.2.2 GetExpectedKernelType的对应关系

</> GetExpectedKernelType对应关系 C++ | 收起 ^

```
1 // 静/老动态图代码
2 framework::OpKernelType GetExpectedKernelType(const framework::ExecutionContext& ctx) const override {
3   auto input_data_type =
4     framework::OperatorWithKernel::IndicateVarDataType(ctx, "X");
```

```
4 // 【省略 mkl_dnn 硬编码的逻辑】
5 return framework::OpKernelType(input_data_type, ctx.GetPlace());
6 }
7
8 // yaml 中的配置
9 kernel :
10     func : cast
11     data_type : x
12
13 // 新动态图 api.cc 中对应的代码
14 kernel_data_type = ParseDataType(x);
```

4.2.3 GetKernelTypeForVar的对应关系

</> GetKernelTypeForVar

C++ | 收起 ^

```
1 // 静态图 and 老动态图代码
2 framework::OpKernelType GetKernelTypeForVar(
3     const std::string& var_name,
4     const phi::DenseTensor& tensor,
5     const framework::OpKernelType& expected_kernel_type) const override
6 {
7     // 不对这两个输入参数进行 DataTransform
8     if (var_name == "SizeTensor" || var_name == "Scale") {
9         return expected_kernel_type;
10    }
11    return framework::OpKernelType(expected_kernel_type.data_type_,
12        tensor.place(), tensor.layout());
13 }
14 // 新动态图中的 "GetKernelTypeForVar" 逻辑
15 PD_REGISTER_KERNEL(bilinear_interp,
16     CPU,
17     ALL_LAYOUT,
18     phi::BilinearInterpKernel,
19     float,
20     double,
21     uint8_t) {
22     // 跳过对于 "SizeTensor" 和 "Scale" 的转换
23     kernel->InputAt(2).SetBackend(phi::Backend::ALL_BACKEND);
24     kernel->InputAt(3).SetBackend(phi::Backend::ALL_BACKEND);
25     // fluid 注册的基本都是ALL_LAYOUT, 因此不跳过 Layout 也没问题
26 }
```

新动态图PrepareData中的GetKernelTypeForVar逻辑

</> 新动态图的PrepareData函数

C++ | 收起 ^

```
1 std::unique_ptr<std::vector<phi::DenseTensor>> PrepareData(
2     const std::vector<Tensor>& inputs,
3     const phi::TensorArgDef& target_args_def,
4     const TransformFlag& transform_flag) {
5     for (const auto& input : inputs) {
6         const auto& tensor_in = input.impl();
7         if (!transform_flag.NeedTransform() || !tensor_in->initialized() ||
8             (!NeedTransformPlace(tensor_in->place(),
9 target_args_def.backend, transform_flag) &&
10             !NeedTransformDataType(tensor_in->dtype(),
11 target_args_def.dtype, transform_flag) &&
12             !NeedTransformLayout(tensor_in->layout(),
13 target_args_def.layout, tensor_in->place(), transform_flag))) {
14             pt_tensors-
15 >emplace_back(*std::dynamic_pointer_cast<phi::DenseTensor>(tensor_in));
16         } else {
17             pt_tensors-
18 >emplace_back(TransformData((static_cast<phi::DenseTensor*>
19 (tensor_in.get()))), target_args_def, transform_flag));
20         }
21     }
22 }
23
24 TransformFlag(bool stop_transform = false,
25               bool trans_dtype = false,    // 此处默认关闭
26               bool trans_backend = true,    // 此处默认打开
27               bool trans_layout = true)    // 此处默认打开
28 : stop_transform_(stop_transform),
29   trans_data_type_(trans_dtype),
30   trans_backend_(trans_backend),
31   trans_layout_(trans_layout) {}
```

phi 中的 kernel 注册：较为复杂，仅了解 ConstructKernel 函数将 kernel 注册到了 KernelFactory

</> ConstructKernel

C++ | 收起 ^

```
1 // paddle/phi/core/kernel_registry.h
2 void ConstructKernel(RegType reg_type,
3                     const char* kernel_name_cstr,
4                     const char* backend_cstr,
5                     DataLayout layout,
```

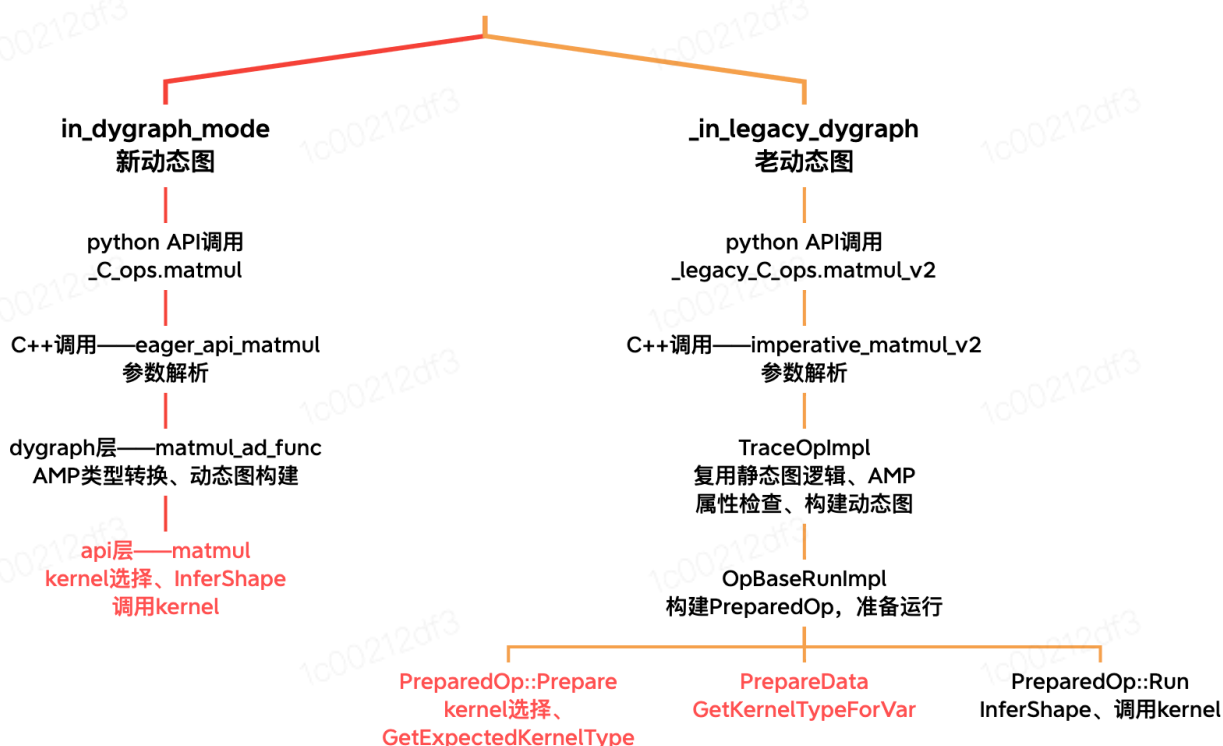
```

6      DataType dtype,
7      KernelArgsParseFn args_parse_fn,
8      KernelArgsDefFn args_def_fn,
9      KernelFn kernel_fn,
10     void* variadic_kernel_fn) {
11     std::string kernel_name(kernel_name_cstr);
12     KernelKey
kernel_key(paddle::experimental::StringToBackend(backend_cstr), layout,
dtype);
13     Kernel kernel(kernel_fn, variadic_kernel_fn);
14     args_parse_fn(kernel_key, kernel.mutable_args_def());
15     args_def_fn(kernel_key, &kernel);
16     if (reg_type == RegType::INNER) {
17         KernelFactory::Instance().kernels()[kernel_name][kernel_key] =
kernel;
18     } else {
19         CustomKernelMap::Instance().RegisterCustomKernel(kernel_name,
kernel_key, kernel);
20     }
21 }

```

4.3 总结

新旧动态图函数调用栈



4.3.1 优点（相比于静/老动态图）

- **性能提升**：解析attr参数时不再采用key:value键值对，而是按照输入顺序进行解析（依赖于代码生成）
- **性能提升**：节省了很多数据结构重复构造的过程（例如Context、Operator）
- **算子复用**：kernel层算子方便复用，可以进行组合运算提升性能（例如 `linear = matmul + add`，减少 dispatch 次数）
- **理解成本**：dygraph、api、kernel三个调用层次清晰，学习成本低
- **高效开发**：大量运用代码生成技术，减少重复的代码copy工作

4.3.2 与静/老动态图的异同

- 静/老动态图的**kernel选择机制**：在 kernel 选择时，未利用layout信息，注册时全部设置为 ALL_LAYOUT
- KernelKey的简化：静/老动态图
 - `class OpKernelType {proto::VarType::Type, DataLayout, Place, LibraryType, customized_type_value_}`
 - `class KernelKey {Backend, DataLayout, DataType}`
- fallback 机制
 - 静 / 老动态图：CUDA 不会 fallback 到 CPU，会直接报错
 - 老动态图：phi 和 fluid 下没有 MKLDNN 的 fallback 逻辑
 - 静态图中：fluid 下 MKLDNN 会 fallback 到 plain cpu；phi 下没有 MKLDNN 的 fallback 逻辑
 - 新动态图：CUDA 会 fallback 到 CPU，无 MKLDNN fallback 逻辑

4.3.3 可能存在的问题

- **性能优化**：kernel选择顺序和次数
 - 新动态图kernel选择时最多需要搜索6次：GPUDNN > GPU > CPU（因为会fallback到 ALL_LAYOUT，3*2=6次）
 - 静/老动态图最多只会选择4次（还是在老动态图兼容新动态图的情况下，phi kernel > fluid kernel）
- **算子复用层次**：算子组合时可以在dygraph、api、kernel三个层次添加，如果不遵守新增算子的原则，未来框架可能变得臃肿
 - PyTorch做到了算子层的复用，paddle实现了kernel层的复用

五、PyTorch dispatch调研

5.1 PyTorch 中的 dispatch 机制

- ✓ PyTorch 没有采取多层的调用逻辑，而是将 Autograd、Tracer、Backend func 的概念放到了同一层次，所有function之间可以相互复用

个人理解：将 Autograd、Tracer 理解为特殊的内部OP，达到了OP间可复用一套代码；而Paddle做到了op和kernel的多层复用

Kernel 选择分发体系对 PyTorch 有重要意义，op层的所有调用都要进行 dispatch

5.1.1 初识 DispatchKey（优先级从低到高）

</> enum class DispatchKey

C++ | 收起 ^

```
1 enum class DispatchKey : uint16_t {
2     Undefined = 0,
3     CatchAll = Undefined,
4     Dense,                // per_backend_func key
5     FPGA,                // 非自定义的backend
6     ORT, Vulkan,
7     Quantized,           // per_backend_func key
8     CustomRNGKeyId,
9     MklDnnCPU,           // mklDnn相关的backend
10    Sparse,               // per_backend_func key
11    SparseCsrCPU,         // 专属于某个 backend 上的特化 func
12    SparseCsrCUDA,
13    NestedTensor,         // per_backend_func key
14    BackendSelect,        // 没有Tensor输入时，无法根据Tensor确定backend，此时调用这个函数确认
15    Python, Fake, FuncTorchDynamicLayerBackMode, Functionalize, Named,
16    Conjugate, Negative, ZeroTensor,
17    ADInplaceOrView,      // 对于 inplace 或者 view, 有特殊的处理
18    AutogradOther,        // 其他 backend 特定的实现
19    AutogradFunctionality, // per_backend_func key
20    AutogradNestedTensor,
21    Tracer,
22    AutocastCPU, AutocastXPU, AutocastCUDA,
23    FuncTorchBatched, FuncTorchVmapMode, Batched, VmapMode,
24    FuncTorchGradWrapper, DeferredInit, PythonTLSSnapshot,
25    FuncTorchDynamicLayerFrontMode, TESTING_ONLY_GenericWrapper,
26    TESTING_ONLY_GenericMode,
27    EndOfFunctionalityKeys, // 用来记录DispatchKeySet最高位的数量
28
29    StartOfDenseBackends,
30    CPU, CUDA, HIP, ...
31    StartOfQuantizedBackends,
32    .....
33    StartOfSparseBackends,
34    .....
35    StartOfNestedTensorBackends,
```

```
32     .....
33     StartOfAutogradBackends,
34     .....
35 }
```

每个 Tensor 都有一个 DispatchKeySet （可以理解为一个bitset），保存 Tensor 中的所有可用 DispatchKey

- DispatchKey 对应位设为 True，表示该函数合法
- 不仅用 enum 列举所有 function，还巧妙的利用了 enum 的优先级，CountLeadingZero 选取优先级最高的 function 调用

❗ 框架内所有的调用function都由 DispatchKey 存储，DispatchKeySet 溢出 uint64_t 应该如何处理？

PyTorch一开始的backend、layout 和 func 是杂糅在 DispatchKey 中的，随着 Dispatch Key 的不断扩展，将 Backend 和 function_key 的概念独立出来

5.1.2 DispatchKey 详谈

</> 14个backend: C10_FORALL_BACKEND_COMPONENTS

C++ | 收起 ^

```
1 #define C10_FORALL_BACKEND_COMPONENTS(_, extra) \
2     _(CPU, extra) \
3     _(CUDA, extra) \
4     _(HIP, extra) \
5     _(XLA, extra) \
6     _(MPS, extra) \
7     _(IPU, extra) \
8     _(XPU, extra) \
9     _(HPU, extra) \
10    _(VE, extra) \
11    _(Lazy, extra) \
12    _(Meta, extra) \
13    _(PrivateUse1, extra) \
14    _(PrivateUse2, extra) \
15    _(PrivateUse3, extra)
```

</> 5个func: C10_FORALL_FUNCTIONALITY_KEYS

C++ | 收起 ^

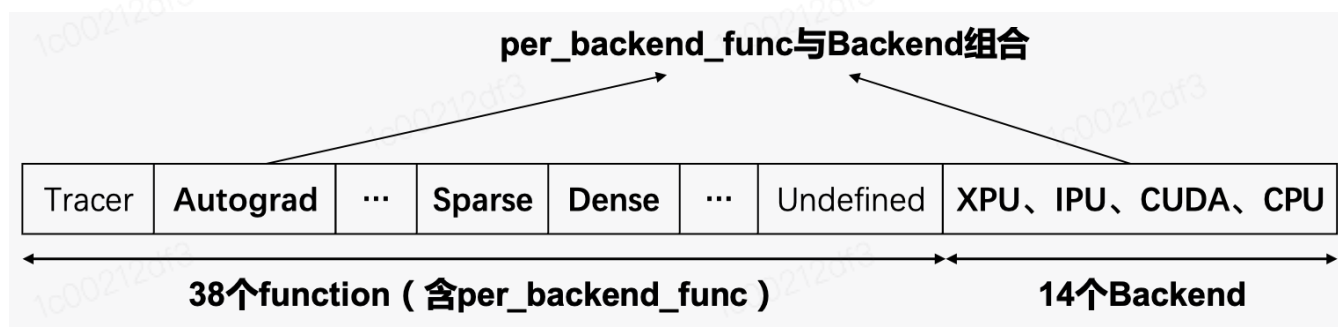
```
1 // 每个 backend 都有这5个特化的 func 实现)
2 // (fullname, prefix)
3 #define C10_FORALL_FUNCTIONALITY_KEYS(_) \
4     _(Dense, ) \
5     _(Quantized, Quantized)
```

```

6  _(Sparse, Sparse)          \
7  _(NestedTensor, NestedTensor) \
8  _(AutogradFunctionality, Autograd)

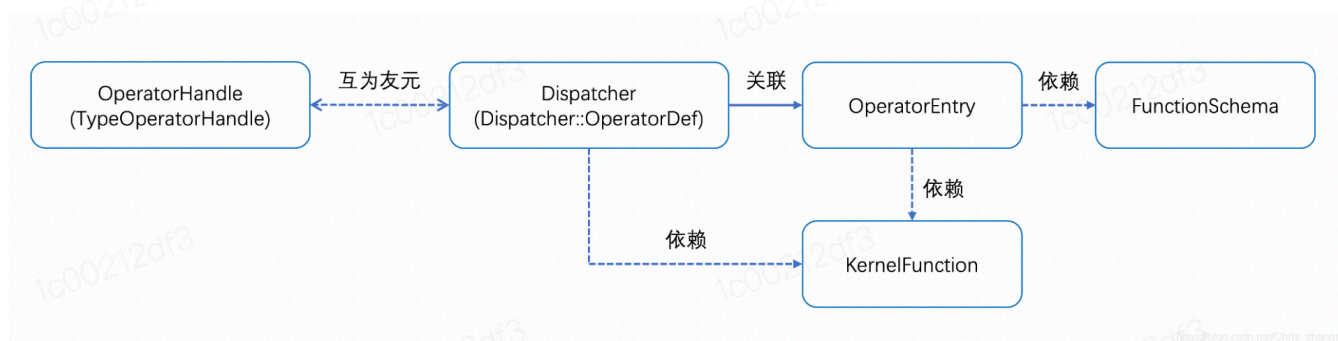
```

DispatchKey = 14 backend * 5 per_backend_func + 38个自定义func/backend, 超出了 `int64_t DispatchKeySet` 可以表示的最大范围, 因此对 `DispatchKeySet` 上的比特位进行二次编码



5.2 dispatch 相关数据结构

5.2.1 类关系描述



5.2.2 OperatorEntry 类：记录算子对应的 dispatchTable_

- 注册/查找 func schema (函数名称、参数)
- 注册/查找 kernel function
- 更新 dispatch table 的相关接口

```
</> class TORCH_API OperatorEntry final
```

C++ | 收起 ^

```

1 // aten/src/ATen/core/dispatch/OperatorEntry.h
2 // Internal data structure that records information about a specific
  operator.
3 // It's not part of the public API; typically, users will interact with
4 // OperatorHandle instead.
5 class TORCH_API OperatorEntry final {
6 private:
7     OperatorName name_;
8     // AnnotatedSchema 由 FunctionSchema schema (op名、参数、返回值) 和
      std::string debug 组成

```

```

9   c10::optional<AnnotatedSchema> schema_;
10  // 解析 schema, 获取里面每个 Tensor的DispatchKeySet 然后取 或 操作, 和
    fallback取 与 操作, 得到真实的 DispatchKeySet
11  // 真实的 DispatchKeySet 包括了, 真实可调用的 kernel
12  DispatchKeyExtractor dispatchKeyExtractor_;
13  // num_runtime_entries 个 KernelFunction, 每个dispatch_key对应一个
    KernelFunction
14  std::array<KernelFunction, c10::num_runtime_entries> dispatchTable_;
15  // 一个 dispatch_key 对应多个 kernel 时 (jupyter cell注册kernel场景下),
    kernels_保存dispatch_key对应的所有kernel, 而dispatchTable_中的dispatch_key只
    能对应一个kernel; dispatchTable_ 中对应的kernel是最新注册的那个, 而kernels_中按
    时间降序排列
16  // dispatchTable[dispatch_key] == kernels_[dispatch_key].front()
17  // AnnotatedKernel = {KernelFunction kernel,
    std::unique_ptr<FunctionSchema> inferred_function_schema, std::string
    debug}. 用户注册的kernel, 相比KernelFunction有更多的信息, 便于调试
18  ska::flat_hash_map<DispatchKey, std::list<AnnotatedKernel>> kernels_;
19 }

```

5.2.3 OperatorHandle/ TypedOperatorHandle 类: 记录op信息, 提供 schema 相关的查询, 提供 kernel 的调用接口

</> class TORCH_API OperatorHandle

C++ | 收起 ^

```

1  // aten/src/ATen/core/dispatch/Dispatcher.h
2  // handle 可以用 dispatcher 来注册 kernel, 或者查找具有特定argument集合的
    kernel
3  class TORCH_API OperatorHandle {
4  // 主要包含成员变量: OperatorEntry op
5  Dispatcher::OperatorDef* operatorDef_;
6
7  // 存储operatorIterator_用来加速 Dispatcher::cleanup() -- program
    termination (and presumably library unloading)时大量运行
8  std::list<Dispatcher::OperatorDef>::iterator operatorIterator_;
9
10 void callBoxed(Stack* stack) const {
11     c10::Dispatcher::singleton().callBoxed(*this, stack);
12 }
13
14 void callBoxedForDispatchKey(DispatchKey dk, Stack& stack) const {
15     c10::Dispatcher::singleton().callBoxedForDispatchKey(*this, dk,
        &stack);
16 }
17

```

```
18 void redispatchBoxed(DispatchKeySet ks, Stack* stack) const {
19     c10::Dispatcher::singleton().redispatchBoxed(*this, ks, stack);
20 }
21 };
```

5.2.4 Dispatcher：组合上面的类功能，实现算子选择分发

- 通过 OperatorEntry 类，注册算子的 schema 和 KernelFunction，更新 dispatch table
- 通过 OperatorHandle 类，使用 KernelFunction 中的 call 函数调用 kernel

</> class Dispatcher

C++ | 收起 ^

```
1 // aten/src/ATen/core/dispatch/Dispatcher.h
2 class TORCH_API Dispatcher final {
3 private:
4     std::list<OperatorDef> operators_;
5     // LeftRight提供了一个线程安全的不用等待的读操作，flat_hash_map是一个高效的
    hash map
6     LeftRight<ska::flat_hash_map<OperatorName, OperatorHandle>>
    operatorLookupTable_;
7     // fallback kernel, 每个DispatchKey对应一个默认的kernel
8     std::array<impl::AnnotatedKernel, num_runtime_entries>
    backendFallbackKernels_;
9 public:
10    static Dispatcher& realSingleton();
11
12    // 查表操作
13    c10::optional<OperatorHandle> findSchema(const OperatorName&
    operator_name);
14    OperatorHandle findSchemaOrThrow(const char* name, const char*
    overload_name);
15    c10::optional<OperatorHandle> findOp(const OperatorName&
    operator_name);
16    const std::vector<OperatorName> getAllOpNames();
17
18    // 调用操作
19    template<class Return, class... Args>
20    Return call(const TypedOperatorHandle<Return (Args...)>& op, Args...
    args) const;
21    template<class Return, class... Args>
22    static Return callWithDispatchKeySlowPath(const
    TypedOperatorHandle<Return (Args...)>& op, at::StepCallbacks&
    stepCallbacks, DispatchKeySet dispatchKeySet, const KernelFunction&
    kernel, Args... args);
23    template<class Return, class... Args>
```

```

24   Return redispatch(const TypedOperatorHandle<Return (Args...)>& op,
DispatchKeySet currentDispatchKeySet, Args... args) const;
25   void callBoxed(const OperatorHandle& op, Stack* stack) const;
26   void callBoxedForDispatchKey(const OperatorHandle& op, DispatchKey dk,
Stack* stack) const;
27   void redispatchBoxed(const OperatorHandle& op, DispatchKeySet
dispatchKeySet, Stack* stack) const;
28
29   // 注册
30   RegistrationHandleRAII registerDef(FunctionSchema schema, std::string
debug, std::vector<at::Tag> tags = {});
31   RegistrationHandleRAII registerImpl(OperatorName op_name,
c10::optional<DispatchKey> dispatch_key, KernelFunction kernel,
c10::optional<impl::CppSignature> cpp_signature,
std::unique_ptr<FunctionSchema> inferred_function_schema, std::string
debug);
32   RegistrationHandleRAII registerName(OperatorName op_name);
33   RegistrationHandleRAII registerFallback(DispatchKey dispatch_key,
KernelFunction kernel, std::string debug);
34   RegistrationHandleRAII registerLibrary(std::string ns, std::string
debug);
35 };

```

IValue 类：Pytorch 一种统一的数据表示，由此也引出了 boxing 和 unboxing 的概念

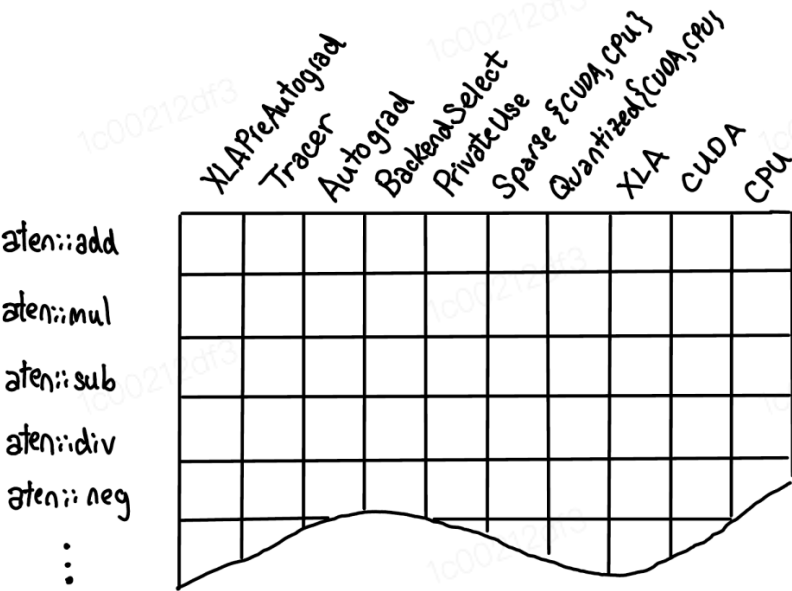
FunctionSchema 类：通过 string 类型的 schema 定义，解析出算子的相关定义，包括输入个数，参数类型及返回值类型等信息。

KernelFunction 类：相当于 std::function，主要是可以从函数指针、仿函数、lambda 函数来构建一个 boxed/unboxed 的 KernelFunction 对象，包括用 boxed/unboxed 不同方式来调用它们，这个类才是最后函数被调用的地方。

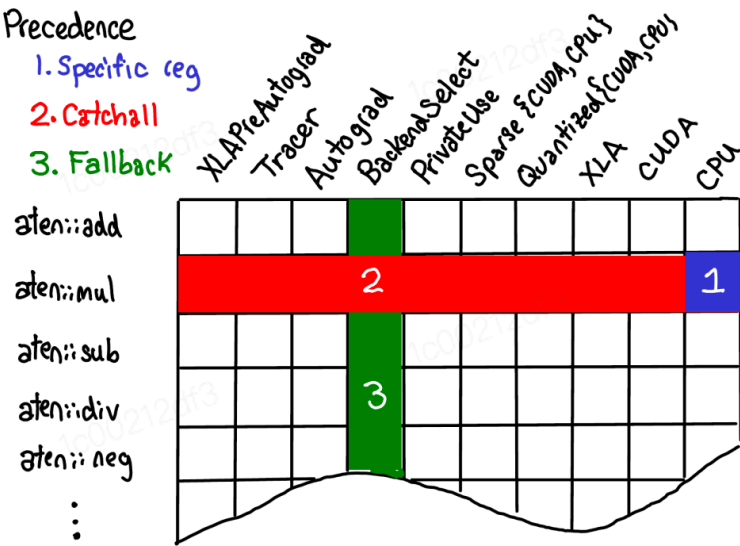
5.3 kernel 注册与调用方式

5.3.1 dispatcher中的表结构

- pytorch中的dispatcher包括Autograd handler、tracing等的选择，进行dispatcher选择时有优先级的概念
- pytorch实际上是一个二维的map，纵轴为op，横轴为dispatch_key（函数和backend）



5.3.2 注册kernel至dispatcher



注册的优先级：1>2>3

特化注册 `m.impl`: 在一个dispatch_key上为某个op注册kernel

</> 特化注册CPU和GPU的kernel C++ | 收起 ^

```
1 TORCH_LIBRARY_IMPL(myops, CPU, m) {
2   m.impl("myadd", myadd_cpu);
3 }
4
5 TORCH_LIBRARY_IMPL(myops, CUDA, m) {
6   m.impl("myadd", myadd_cuda);
7 }
8
9 TORCH_LIBRARY_IMPL(myops, Autograd, m) { // 不需要autograd的op
10  m.impl(op, autogradNotImplementedFallback());
11 }
```

```
12 TORCH_LIBRARY_IMPL(myops, ADInplaceOrView, m) { // inplace或者view操作的op
13     m.impl(op, autogradNotImplementedInplaceOrViewFallback());
14 }
```

catch-all注册 m.def: 定义op的schema

</> catch-all的注册方式

C++ | 收起 ^

```
1 TORCH_LIBRARY(aten, m) {
2     m.def("myadd(Tensor self, Tensor other) -> Tensor");
3 }
```

fallback注册 m.fallback: 为所有op定义默认的处理逻辑

</> fallback注册默认kernel

C++ | 收起 ^

```
1 TORCH_LIBRARY_IMPL(_, Tracer, m) {
2     m.fallback(generic_tracing);
3 }
```

5.4 Paddle 与 PyTorch 的比较

- 选择分发体系的不同

paddle: 每个op对应多个backend的kernel，最多对应 {backend, layout, datatype} 笛卡尔积个kernel

pytorch: op 是一个更广泛的概念（计算接口），例如 autograd 也算一个“op”，选择时从 {backend, layout}中选择，datatype通过switch-case处理

- Pytorch将选择分发提到了更加全局的位置，大大加强了可复用性（不仅是kernel可复用，Autograd等也可复用）
 - 缺点：dispatch的过程增大了调度的开销，尤其是调用一个算子，需要多次dispatch

六、Kernel 选择分发优化方案

- Tips:** **MKL-DNN**是 intel 发布的用于深度神经网络的英特尔数学核心库，随着 **oneAPI** 的启动，项目名称更改为 **oneDNN**(one Deep Neural Network)
mkldnn 是在 CPU 上专用于深度神经网络的加速库；cudnn 是在 GPU 上专用于深度神经网络的加速库

6.1 背景

去除通过 use_mkldnn, use_cudnn 等硬属性的方式异化 kernel 选择，设计通用的选择机制；
长期统一 yaml kernel 选择和原 Op GetExpectedKernelType 的体系，目前是冗余的；

对应到《2022 Q4工作规划》

Kernel选择分发体系的优化与统一（Q3以方案调研为主）

1. 现状：kernel选择通过传入use_mkldnn, use_cudnn这种后门参数指定，phi和fluid kernel选择体系是两套
 2. 性质：必要机制，设计难度高，涉及全量op，统一工作量大
 3. 人力：假设Q3能够产出方案，Q4需要1人 * 2月，要求该同学对框架架构熟悉
- use_mkldnn进行硬属性kernel选择——设计硬编码优化方案并实现
 - #ifdef PADDLE_WITH_MKLDNN 硬编码耦合入了框架的每个部分，后期重构和维护成本极高
 - 为了适配少数算子，在几乎所有 mkldnn 的 op 中嵌入硬编码
 - 部分算子当初未采用优雅的实现方式（将 mkldnn 注册为 custom 算子）
 - 历史原因：当时的开发者没有架构意识，以功能实现为主
 - 最终统一 fluid 和 phi 的选择体系——如何拆分执行路径、每个路径的执行方案？
 - 梳理两个选择体系的问题
 - fluid 选择体系有多个硬编码，#ifdef PADDLE_WITH_MKLDNN, PADDLE_WITH_XPU.....，phi下如何最大限度的避免？
 - phi 下如何支持 mkldnn、gpudnn？——目前新动态图还无法直接调用 phi 下的 mkldnn kernel，走老动态图的路径调用 phi
 - phi 的选择体系中，KernelKey 有 Backend 的概念，然而传参 Tensor 只有 place 信息
 - 两者的选择体系如何统一？
 - 总原则：后续老动态图退场明确的背景下，应该让老动态图的行为逻辑向新动态图对齐
 - yaml配置、代码生成、可变输入参数支持

6.2 执行路径

 加载中...

问题分析：去掉硬编码 和 选择体系的统一 实际上是两个耦合的子工作。

- 直接统一选择体系（收益高、难度大、耗时长），将fluid的选择体系迁移到phi后，其实相当于绕过了GetExpectedKernelType，根本不需要去掉硬编码，可以直接完成。

i fluid 和 phi 分发体系的统一，依赖的前置流程较多，fluid 下硬编码优化的工作可并行开展

去除mkldnn硬编码，有利于梳理现有kernel选择逻辑，辅助phi下对mkldnn backend的支持

6.2.1 fluid 下的执行路径

1. **【已完成】** 常规 fluid mkldnn op （约占整体的 60%）中 mkldnn 硬编码的移除
2. **【进行中】** 剩余算子逐个分析并解决，可分成几大类问题，见下文

6.2.2 phi 下的执行路径

1. **【已完成、日升】** 支持Kernel注册时指定准确的Layout，现在mkldnn注册只能将layout注册为ALL_LAYOUT，支持注册成特定的DataLayout::ONEDNN
2. 新动态图不再fallback至ALL_LAYOUT，现在每个backend都需要重复搜索一次ALL_LAYOUT的kernel，优化现有dispatch逻辑，优化搜索kernel性能
3. 新动态图支持mkldnn和cudnn backend，现在新动态图无法支持调用mkldnn kernel
 - a. 调研竞品实现，学习竞品如何传入use_cudnn/mkldnn信息，尝试优化现有kernel选择方案

6.3 硬编码优化方案

6.3.1 代码梳理

i GetExpectedKernelType 是 #ifdef PADDLE_WITH_MKLDNN 的重灾区，因此优先对该函数进行分析和硬编码移除

常规 fluid mkldnn op （约占整体的 60%）

</> 常规 fluid mkldnn op

C++ | 收起 ^

```
1 #ifdef PADDLE_WITH_MKLDNN
2 #include "paddle/fluid/platform/mkldnn_helper.h"
3 #endif
4
5 // 设置 DataLayout & LibraryType
6 class AbsOp : public framework::OperatorWithKernel {
7     auto input_data_type =
8         framework::OperatorWithKernel::IndicateVarDataType(ctx, "X");
9 #ifdef PADDLE_WITH_MKLDNN
```

```

9    // 一定要有 use_mkldnn 属性才需要做转换
10   if (this->CanMKLDNNBeUsed(ctx, input_data_type)) { // ctx.Attr<bool>
    (use_mkldnn) 且 is_cpu_place
11       return framework::OpKernelType(input_data_type,
12                                       ctx.GetPlace(),
13                                       framework::DataLayout::kMKLDNN,
14                                       framework::LibraryType::kMKLDNN);
15   }
16 #endif
17   return framework::OpKernelType(input_data_type, ctx.GetPlace());
18 }
19

```

op 特例类型1——添加强约束（18个op）

</> op 特例类型1——添加强约束

C++ | 收起 ^

```

1 // pad_2d, pad_3d; split
2 #ifdef PADDLE_WITH_MKLDNN
3     // only constant mode and non-blocked layouts are supported for
    oneDNN
4     if (this->CanMKLDNNBeUsed(ctx, input_data_type) &&
5         ctx.Attr<std::string>("mode") == "constant" &&
6         ctx.Input<Tensor>("X")-
    >mem_desc().data.format_desc.blocking.inner_nblks == 0) {
7         return framework::OpKernelType(input_data_type,
8                                         ctx.GetPlace(),
9                                         framework::DataLayout::kMKLDNN,
10                                         framework::LibraryType::kMKLDNN);
11     }
12 #endif

```

op 特例类型2——具有customize_type_value（11个op）

</> op 特例类型2——具有customize_type_value

C++ | 收起 ^

```

1 class AddMMOp : public framework::OperatorWithKernel {
2     framework::OpKernelType GetExpectedKernelType(const
    framework::ExecutionContext& ctx) const {
3         auto input_data_type = OperatorWithKernel::IndicateVarDataType(ctx,
    "X");
4 #ifdef PADDLE_WITH_MKLDNN
5     if (this->CanMKLDNNBeUsed(ctx, input_data_type)) {
6         if (input_data_type ==
            framework::DataTypeTrait<int8_t>::DataType()) ||

```

```

7         input_data_type ==
framework::DataTypeTrait<uint8_t>::DataType()) {
8         customized_type_value = kMULMKLDNNINT8;
9     }
10    return framework::OpKernelType(input_data_type,
11                                   ctx.GetPlace(),
12                                   framework::DataLayout::kMKLDNN,
13                                   framework::LibraryType::kMKLDNN,
14                                   customized_type_value);
15    }
16 #endif
17    return framework::OpKernelType(input_data_type, ctx.GetPlace());
18 }
19 };

```

op 特例类型3——仅注册了mkldnn kernel，但是未实现GetExpectedKernelType（12个op，squeeze、reshape、flatten）

squeeze 依赖于 intel 推进，支持该功能或者删除mkldnn kernel，reshape、flatten 确认是否需要设置 MKLDNN_layout

</> op 特例类型3——仅注册了mkldnn kernel

C++ | 收起 ^

```

1  framework::OpKernelType GetExpectedKernelType(
2      const framework::ExecutionContext &ctx) const override {
3      auto input_data_type =
4          framework::OperatorWithKernel::IndicateVarDataType(ctx, "X");
5
6      // #ifdef PADDLE_WITH_MKLDNN
7      //     if (this->CanMKLDNNBeUsed(ctx, input_data_type)) {
8      //         return framework::OpKernelType(input_data_type,
9      //                                         ctx.GetPlace(),
10         //
11         framework::DataLayout::kMKLDNN,
12         //
13         framework::LibraryType::kMKLDNN);
14     //     }
15     // #endif
16     return framework::OpKernelType(input_data_type, ctx.GetPlace());
17 }

```

✅ 在代码重构的过程中，原有逻辑不会消失，无非是逻辑迁移或者逻辑覆盖

6.3.2 硬编码外移（已完成）

在 `GetExpectedKernelType` 函数调用之后，立即修改 `DataLayout::kMKLDNN` 和 `LibraryType::kMKLDNN`，将 `#ifdef PADDLE_WITH_MKLDNN` 的逻辑外移

优点：实现较为简单，耗时短

缺点：降低 fluid 下的代码可读性（后续可以考虑优化可读性）；部分 `mkldnn op` 的特例仍然需要保留硬编码

考虑：特例几乎都包含更严格的 `mkldnn` 判断条件（更强的约束），需要一种机制，从 `GetExpectedKernelType` 中获取bool信息（是否满足强约束）——或者需要一种白名单机制，对特例 `op` 做豁免

6.3.3 硬编码下沉（进行中）

每个被硬编码的op，都对应一个`mkldnn kernel`，可以在 `kernel` 内部 `transform layout`

- 注意：实际上 `DataTransform` 应该是框架做的事情，而非 `kernel` 的功能
- 改进：对特例的 `mkldnn op`，实现硬编码的下沉，例如 `customize_value` 的 `op`，其实并不需要自定义值

优点：可以解决特例的 `corner case`

缺点：工作量较大；几十个 `kernel` 里面有相同的逻辑

6.4 新动态图支持 `mkldnn kernel`

问题核心：新动态图api层的调用，参数通过Tensor传入，然而Tensor中仅有Place信息，Backend是Place信息的超集，如何在Tensor中加入Backend相关的内容？

问题约束1：没有必要为了 `mkldnn` 和 `cudnn`，在 `Tensor` 中加入 `Backend`，改动较重

问题约束2：新动态图设计之初，就尽量避免通过全局 `flag` 传入信息

问题约束3：希望给用户自行提供自行选择，是否使用加速库的可能

备选方案1

- 为Tensor添加bool型字段：`is_use_dnn`，默认为True，如果编译了MKLDNN / CUDNN，在 `kernel`选择时默认选择 `mkldnn/cudnn`
- 涉及到的改动内容：
 - 修改`kernel`选择逻辑；
 - 修改python端Tensor定义；
 - 注意：什么时候给 `Tensor` 赋值 `is_use_dnn`？

其他方案（需持续调研）

七、个人工作和感悟

7.1 个人工作

7.1.1 算子迁移与机制完善 (0718-0901)

共计完成 16 个算子的迁移（9 个特殊算子、2 个延期算子、5 个XPU算子）

- 完成 **softsign** 特殊算子迁移，解决命名空间冲突的问题。[PR44616](#)
- 完成 **fill_any** 特殊算子迁移，解决输入参数个数不匹配的问题。[PR44879](#)
- 完成 **interpolate** 系列五个特殊算子迁移，完善 optional<vector<Tensor>> 输入的算子迁移机制。[PR45072](#)、[PR45140](#)、[PR45145](#)、[PR45148](#)、[PR45151](#)
- 完成 **merged_momentum** 和 **merged_adam** 两个优化器相关的延期算子迁移，完善 inplace optional vector 组合输入的算子库机制。[PR45359](#)、[PR45367](#)
- 完成 **coalesce_tensor** 特殊算子迁移，修复编译依赖与单测错误。[PR45478](#)
- 完成 **memcpy** 特殊算子迁移，修复 Alloc 无法将 GPU Tensor 分配至 GPUPinnedPlace 的问题。[PR45622](#)
- 完成 5 个 XPU 算子迁移：truncated_gaussian_random [PR45529](#)、update_loss_scaling 和 check_finite_and_unscale [PR45569](#)、coalesce_tensor [PR45579](#)、uniform_random [PR45583](#)

7.1.2 kernel 选择分发相关工作 (0902 至今)

- **mkldnn硬编码移除**（部分 PR 列举如下）
 - 去除 GetExpectedKernelType 中的 mkldnn 硬编码，解决了 60%左右 mkldnn op中硬编码的问题。[R46606](#)
 - 整理统一 28个 op 中的 GetExpectedKernelType 实现，便于后续去除硬编码的工作。[PR46053](#)、[PR46163](#)、[PR46666](#)、[PR46677](#)、[PR46681](#) 等PR
 - 移除 7 个 op 中的无用 mkldnn 代码。[PR46660](#)、[PR46676](#)、[PR46690](#) 等PR
 - 去除 interp、transpose2 中的特化 mkldnn 逻辑。[PR46685](#)、[PR46692](#)
 - 修复 test_dot_op 单测中对 use_mkldnn 属性的使用。[PR46848](#)
- **优化现有的 kernel 选择分发体系**
 - 优化backend优先级，GPUDNN > GPU > ONEDNN > CPU。[PR46526](#)
 - kernel选择时，使用优先级最高的layout。[PR46598](#)
 - BackendSet从 uint64_t 优化为 uint32_t。[PR46532](#)
 - 修复KernelKeyParser中operator动作不统一的问题，并添加了类型检查。[PR46560](#)
 - 修复 phi 下对 mkldnn kernel 的 layout 注册方式。[PR46626](#) - [PR46629](#)

7.2 感悟

- **框架开发要守原则**：很多设计最初都是好的设计，但是中间遇到问题时会不断的妥协，导致框架变得冗杂（例如 kernel 选择时直接用MKLDNN 硬编码）
- **框架开发要勤同步**：有些代码已经考虑到为以后留出了接口，但是其他人不了解的话，很容易用别的方式实现，污染框架（例如op_registry 中，mkldnn 注册时已经同步了 layout 和 library_type ）；自身也要常读代码，常读常新

- 框架开发要有边界：如何实现一个功能有很多种方案、可以在很多地方实现。仅仅实现是不够的，需要遵守每个模块的边界（例如 kernel 选择体系中的 DataTransform 是框架做的事情，就不能交给 Kernel 来做）

八、现场讨论问题整理与答复

- kernel选择方案支持之后，静态图也是都支持的吗？动静一致方面有何考虑？@留杰

kernel选择分发体系的最终目标是，统一静态图/老动态图/新动态图的选择分发逻辑。因此kernel选择方案支持后，静态图也是支持的，并且动静态图之间的选择动作相同。

当前状态：正在清理kernel选择过程中，每个op `GetExpectedKernelType` 函数中的mkldnn硬编码，这部分op很多都是静态图和老动态图共用的op，因此这部分的清理过程首先是一致的

后续的考虑：静态图的kernel选择逻辑向新动态图的 `yaml` 靠拢，最终使得动静态图在选择kernel时以 `yaml` 为标准

另外，目前框架内静态图和老动态图的选择逻辑其实是有稍许偏差的，例如静态图实际上没有找到mkldnn的kernel时，会fallback到CPU的plain kernel上，而目前老动态图并没有这一机制。在后续的实现过程中我也会注意fallback的情形。

- Tensor中的MKLDNN有了解过吗？为什么不能新建一个MKLDNN的Tensor？@红雨

当前 `phi` 下的 `DenseTensor` 类有一段被 MKLDNN hack 的逻辑，加入了对 MKLDNN tensor 格式的描述，从这个 Tensor 也可以看出来，MKLDNN 的硬编码几乎已经侵入了框架的每个层次。

新建一个MKLDNN Tensor指的是，可以新建一个MKLDNN类，继承自DenseTensor，在这个MKLDNN Tensor中持有特化的信息，而不污染 paddle 内的其他Tensor。出于如下考虑，是没有办法实现的：

1. 转化或者新建一个MKLDNN Tensor，需要知道选择出来的kernel是一个MKLDNN kernel；然而kernel的选择过程，也依赖于Tensor中持有MKLDNN的信息，这就变成了一个先有鸡还是先有蛋的问题。
2. 如果加入MKLDNN的Tensor，要在所有需要转化的位置加上static_cast，这样做的改动成本过高，同时框架的性能也可能受影响。

- 框架内 fallback 的实现机制？@陈卓

Paddle 目前在kernel选择分发时，遵循着如下的选择优先级：phi kernel > fluid kernel；Xpu kernel > CPU kernel（此处的Xpu指除了CPU以外的所有硬件）fallback指的是在kernel选择时，如果没有找到Xpu的kernel，则会fallback到CPU的kernel，保障代码的正常运行。

然而并不是所有的Xpu都可以fallback到CPU中，特例是GPU的kernel不会fallback到CPU，如果GPU的kernel没有找到，则会直接报错。从代码调试的角度出发，如果支持GPU fallback 到CPU，那么当模型的性能下降时，会很难排查，需要确定有没有找到对应kernel，还是其他的问题。

- torch dispatch key和我们对比的优劣？structured kernel的了解？@嘉彬

torch 的选择分发机制和 paddle 的选择分发对比，有下列的优点：

1. 可复用性高，torch 把选择分发提升到了一个更加全局的角度。torch 相当于把 autograd、Trace 这些更上层的函数，和 kernel 放到了同一层次，也仅有这一层次，所有函数都可以相互复用；而 paddle 的调用层次有三层：dygraph 层、api 层和 kernel 层，不同层次的函数间无法相互调用，因此对于 paddle 来说，如何决定函数在那一层复用，是未来需要着重注意的问题，否则容易造成框架不同层次的耦合。
2. torch 的这套选择分发机制，蕴含了很多 trick 在里面。例如 dispatchKey 的 enum class 不仅利用了枚举值，还利用枚举值的大小蕴含了优先级的概念，优先级也可以理解为调度顺序。paddle 对于 mklDnn 处理时，将 mklDnn 硬编码嵌入到各个层次，而 pytorch 就可以把 mklDnn 的实现都放到一个 dispatchKey 上，可扩展性更强。

缺点也很明显：

1. 所有函数调用的时候都要走一遍 dispatch 逻辑，当调用栈较深时，dispatch 的选择开销过重，影响框架性能。
2. dispatch 内的概念相对来说更混乱，糅合了 autograd、kernel、backend 等等信息，对外部开发者不友好。

structured kernel 可能是 torch 解决 dispatch 性能的一种思路，对我个人而言，还没有开发场景需要了解 torch 的 structured kernel 机制，因此还没有深入了解过 structured kernel，未来可能会深入调研。

- **torch 在 dispatch 时怎么解决多 kernel 选择问题？dispatch 怎么选择 BLD 三元组？pytorch 注册时候的用时为什么短？@王欢**

首先 torch 中的 layout 概念与 paddle 是不同的，paddle 的 layout 包括 sparse/dense、NCHW/NHWC 这两类信息，实际上这两类不应该放到一起组织；torch 中的 layout 仅指 sparse/dense，因此 layout 的数量远小于 paddle。

1. torch 在 dispatch 时，将 backend 和 layout 组合成一个 DispatchKey，例如有 DenseCPU、DenseGPU、SparseCPU 等，当选择到对应的 DispatchKey 时，则会执行对应 backend 和 layout 的 kernel
2. torch 在 dispatch 时，仅选择了 backend 和 layout，选择 datatype 的逻辑封装在 kernel 中，相当于 kernel 内部用 switch-case 语句来判断，执行哪个特化逻辑。
3. 当运行 import paddle 或者 import pytorch 时，执行框架注册 kernel 的逻辑，pytorch 注册用时短，可能是因为 torch 注册的 kernel 数量少。因为 torch 只需要注册 backend * layout 个 kernel，而 paddle 需要注册 backend * layout * datatype 个 kernel。paddle 现有的注册方式有历史原因在里面。

